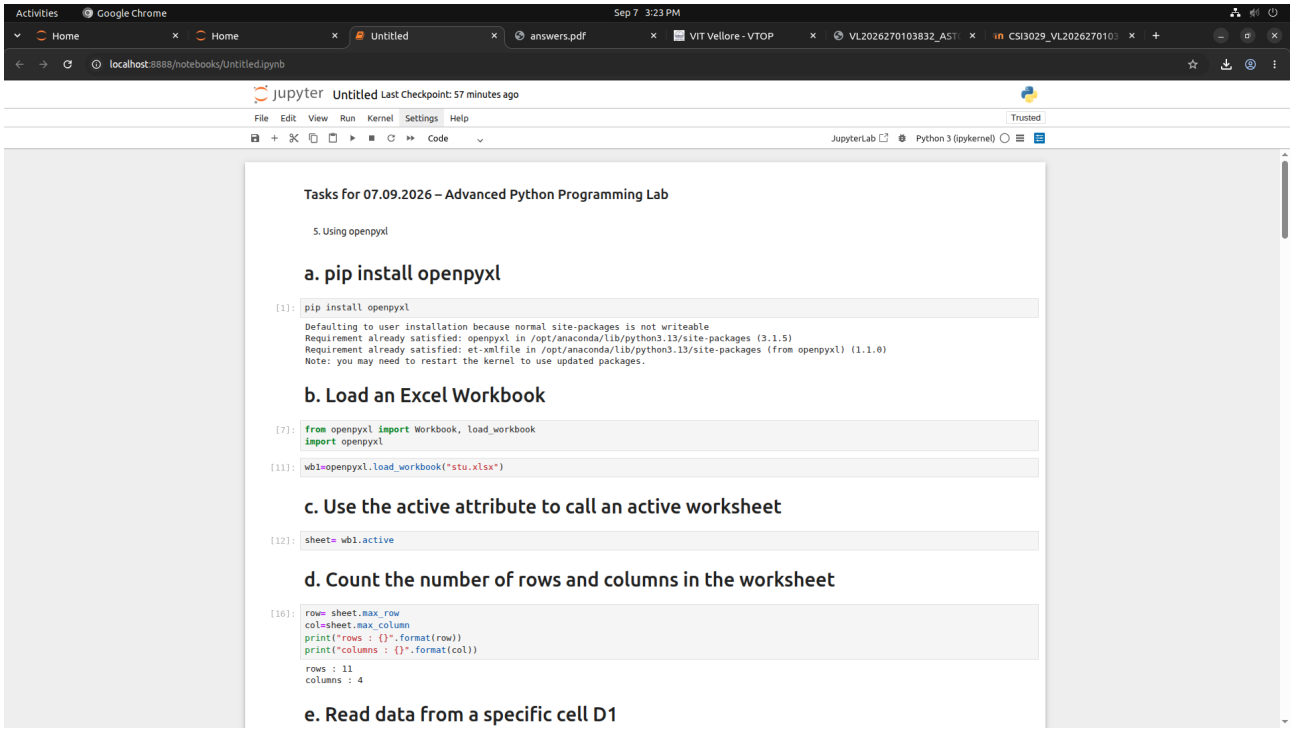
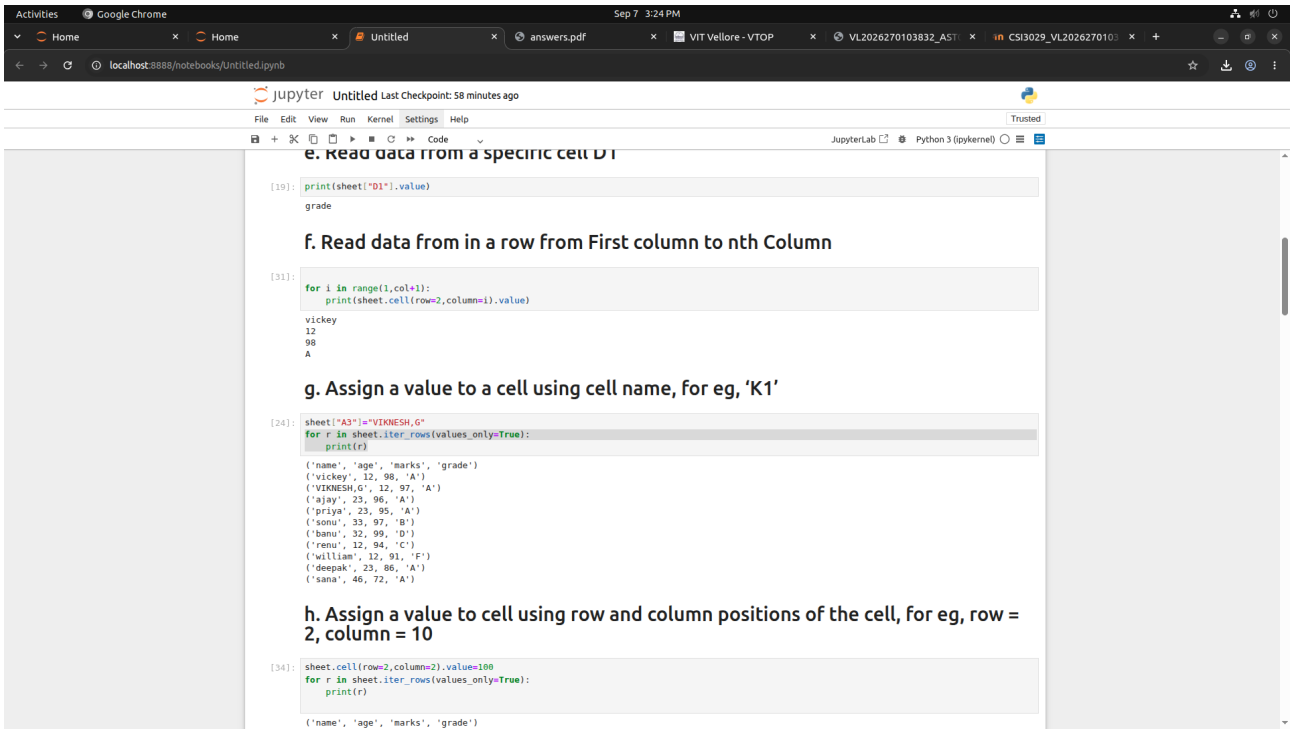
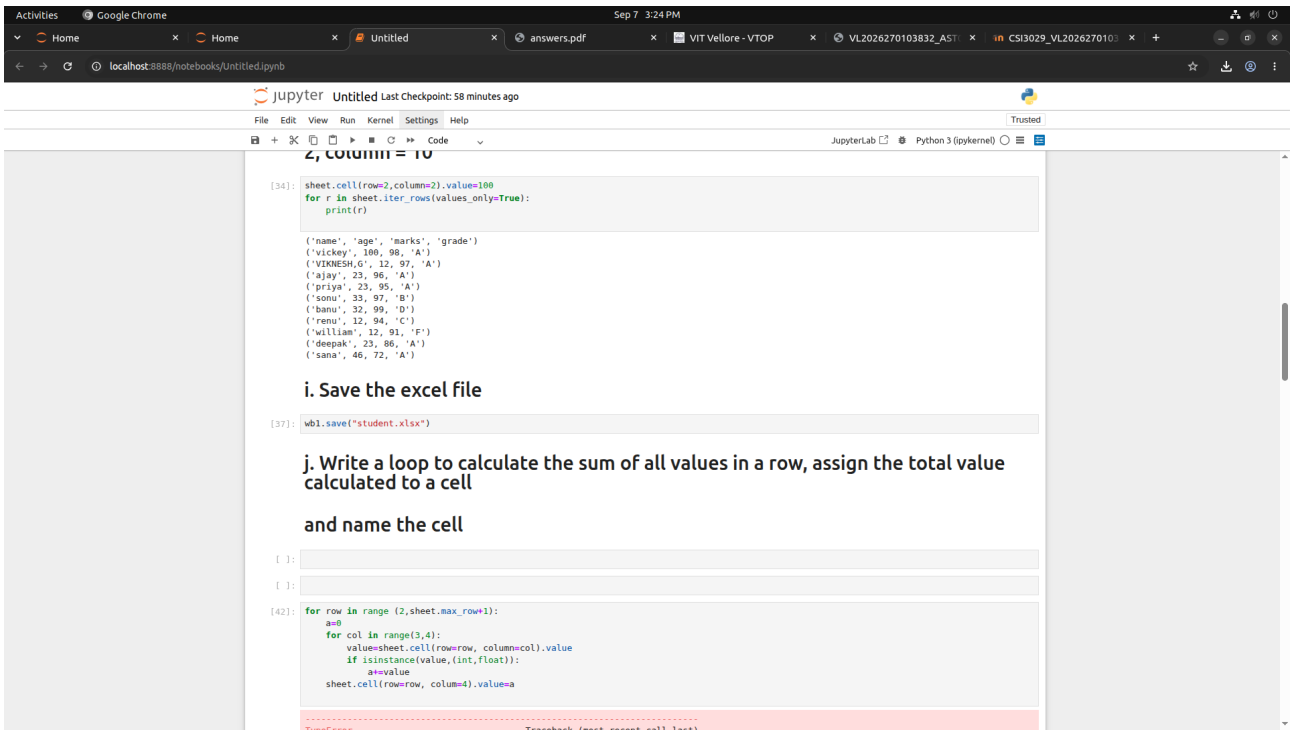
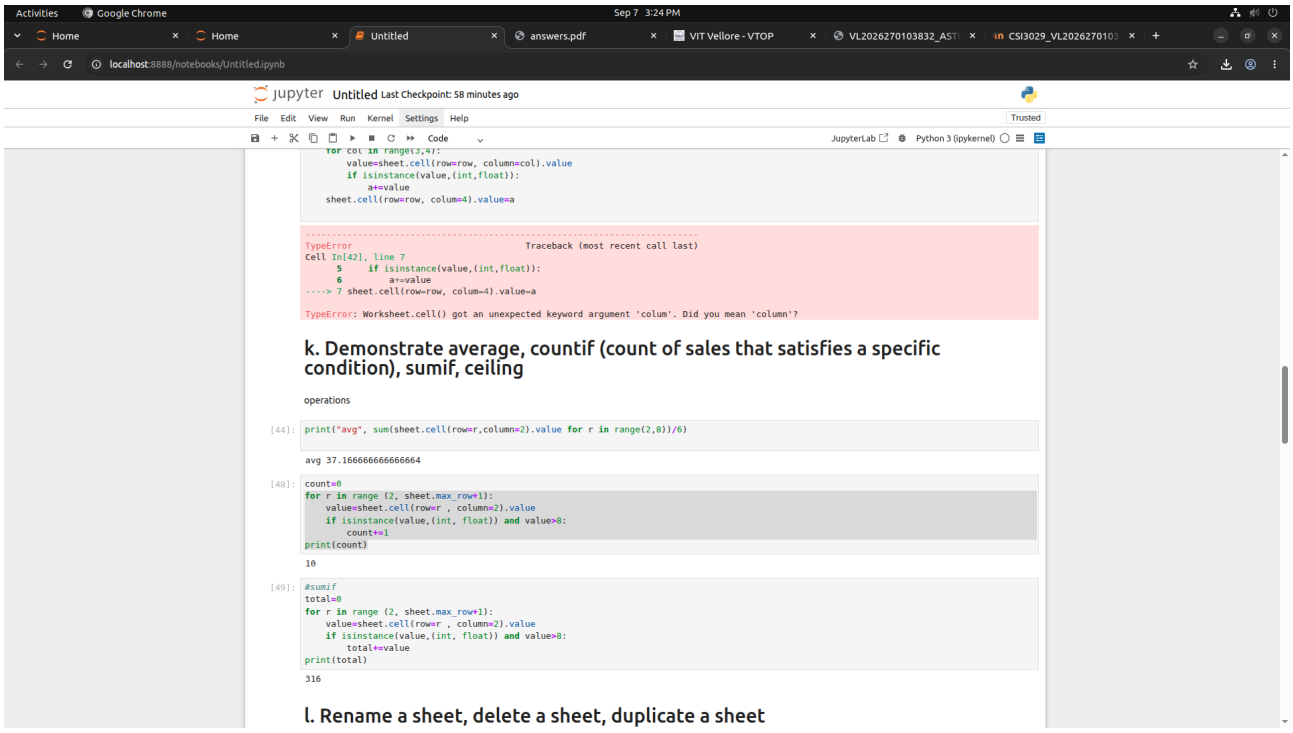
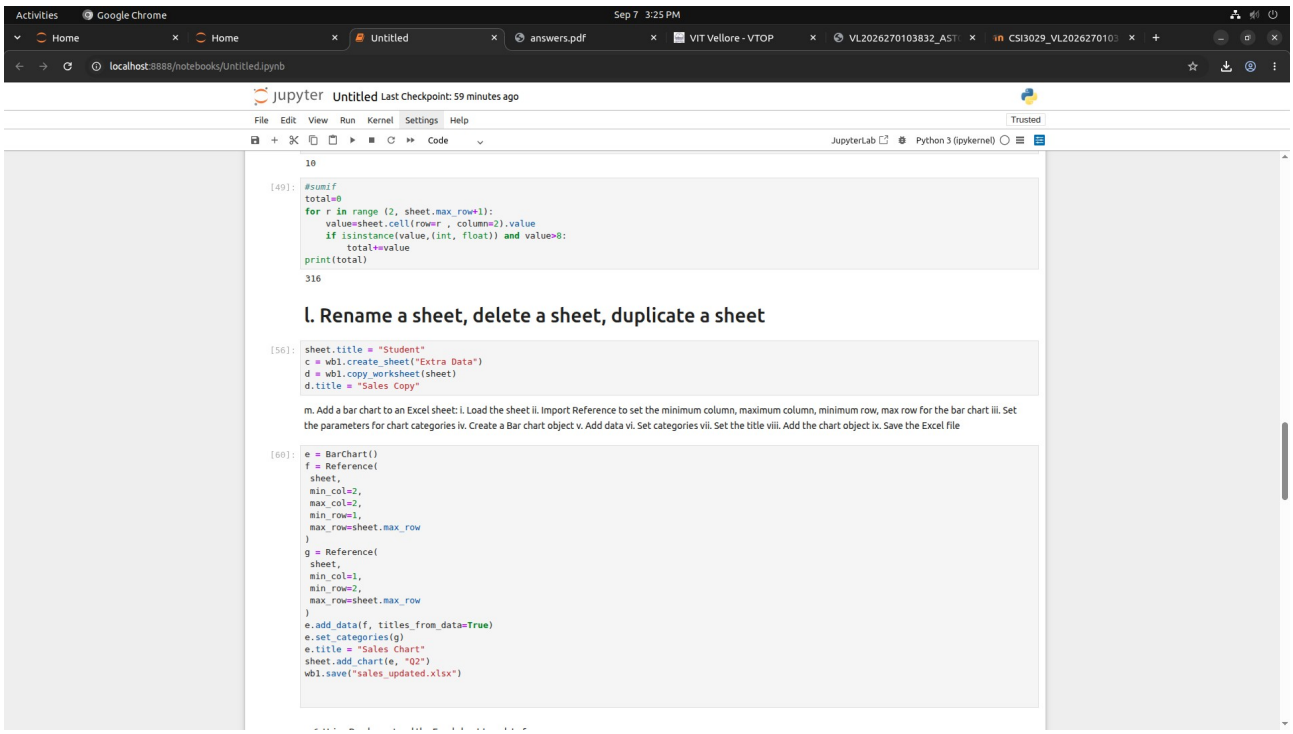
NAME:VIKNESH.G
REG.NO: 23MID0161

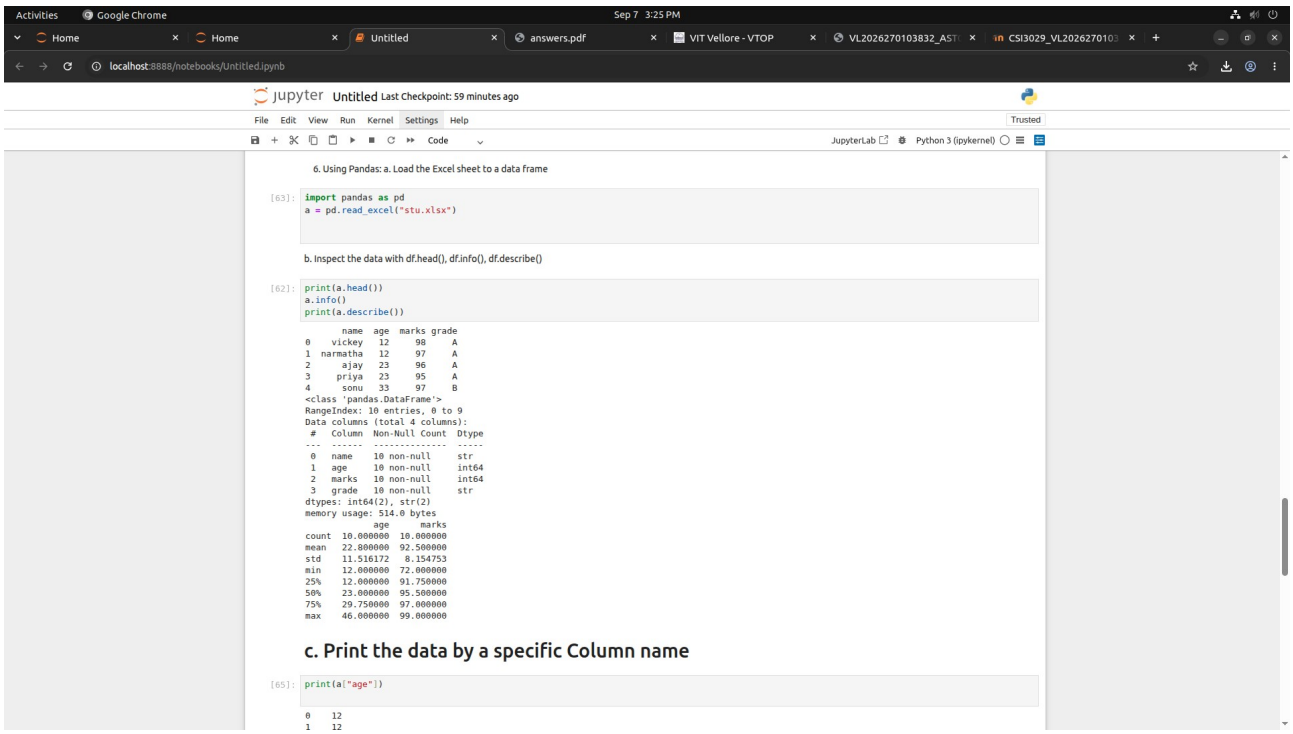


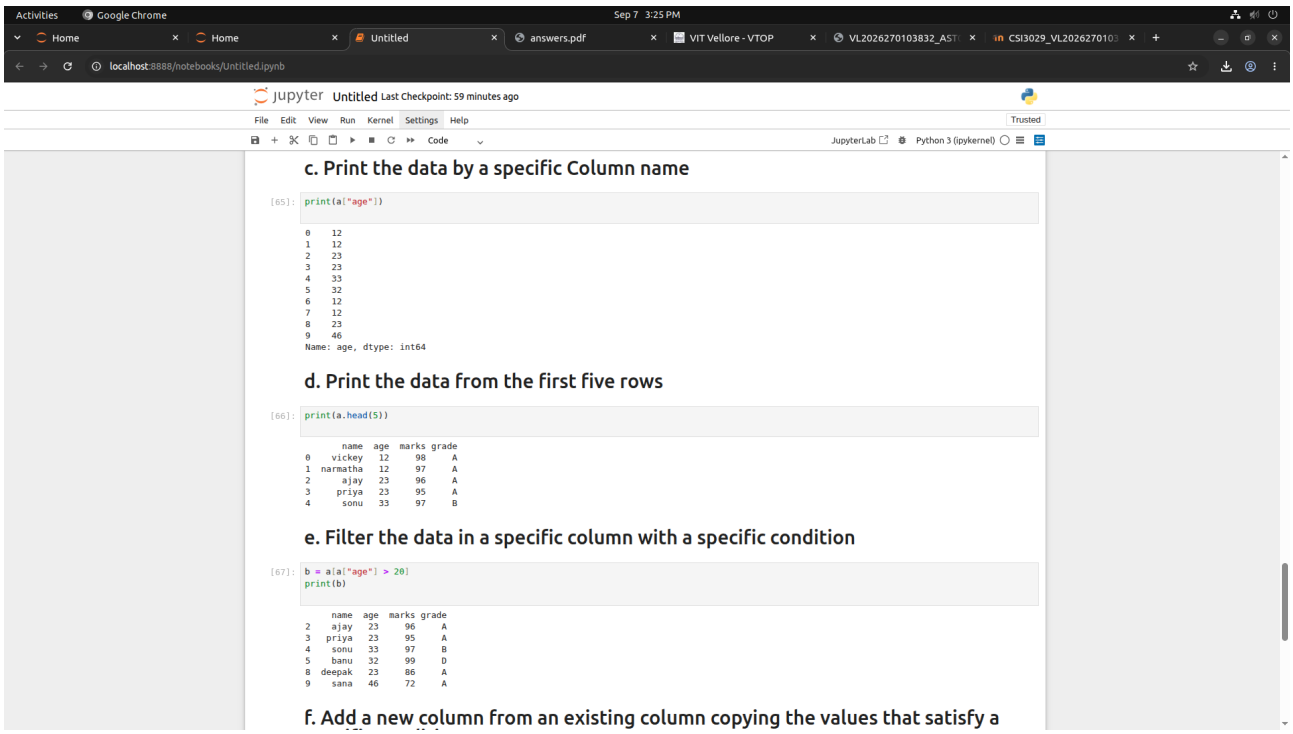



Activities
Google Chrome
Sep 7 3:24 PM
Home
Home
Untitled
answers.pdf
VIT Vellore - VTOP
localhost:8888/notebooks/Untitled.ipynb
jupyter Untitled Last Checkpoint: 58 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
2, column = 10
[34]:
sheet.cell(row=2,column=2).value=100
for r in sheet.iter_rows(values_only=True):
print(r)
('name', 'age', 'marks', 'grade')
('vickey', 100, 98, 'A')
('VIKNESH,G', 12, 97, 'A')
('ajay', 23, 96, 'A')
('priya', 23, 95, 'A')
('sonu', 33, 97, 'B')
('banu', 32, 99, 'D')
('renu', 12, 94, 'C')
('william', 12, 91, 'F')
('deepak', 23, 86, 'A')
('sana', 46, 72, 'A')
i. Save the excel file
[37]:
wb1.save("student.xlsx")
j. Write a loop to calculate the sum of all values in a row, assign the total value calculated to a cell
and name the cell
[ ]:
[ ]:
[42]:
for row in range (2,sheet.max_row+1):
a=0
for col in range(3,4):
value=sheet.cell(row=row, column=col).value
if isinstance(value,(int,float)):
a+=value
sheet.cell(row=row, colum=4).value=a


Activities
Google Chrome
Sep 7 3:24 PM
Home
Home
Untitled
answers.pdf
VIT Vellore - VTOP
localhost:8888/notebooks/Untitled.ipynb
jupyter Untitled Last Checkpoint: 58 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
for col in range(3,4):
value=sheet.cell(row=row, column=col).value
if isinstance(value,(int,float)):
a+=value
sheet.cell(row=row, colum=4).value=a
TypeError Traceback (most recent call last)
Cell In[42], line 7
5 if isinstance(value,(int,float)):
6 a+=value
----> 7 sheet.cell(row=row, colum=4).value=a
TypeError: Worksheet.cell() got an unexpected keyword argument 'colum'. Did you mean 'column'?
k. Demonstrate average, countif (count of sales that satisfies a specific condition), sumif, ceiling
operations
[44]:
print("avg", sum(sheet.cell(row=r,column=2).value for r in range(2,8))/6)
avg 37.166666666666664
[48]:
count=0
for r in range (2, sheet.max_row+1):
value=sheet.cell(row=r , column=2).value
if isinstance(value,(int, float)) and value>8:
count+=1
print(count)
10
[49]:
#sumif
total=0
for r in range (2, sheet.max_row+1):
value=sheet.cell(row=r , column=2).value
if isinstance(value,(int, float)) and value>8:
total+=value
print(total)
316
l. Rename a sheet, delete a sheet, duplicate a sheet


Activities
Google Chrome
Sep 7 3:25 PM
Home
Home
Untitled
answers.pdf
VIT Vellore - VTOP
localhost:8888/notebooks/Untitled.ipynb
jupyter Untitled Last Checkpoint: 59 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
10
[49]:
#sumif
total=0
for r in range (2, sheet.max_row+1):
value=sheet.cell(row=r , column=2).value
if isinstance(value,(int, float)) and value>8:
total+=value
print(total)
316
l. Rename a sheet, delete a sheet, duplicate a sheet
[56]:
sheet.title = "Student"
c = wb1.create_sheet("Extra Data")
d = wb1.copy_worksheet(sheet)
d.title = "Sales Copy"
m. Add a bar chart to an Excel sheet: i. Load the sheet ii. Import Reference to set the minimum column, maximum column, minimum row, max row for the bar chart iii. Set the parameters for chart categories iv. Create a Bar chart object v. Add data vi. Set categories vii. Set the title viii. Add the chart object ix. Save the Excel file
[60]:
e = BarChart()
f = Reference(
sheet,
min_col=2,
max_col=2,
min_row=1,
max_row=sheet.max_row
)
g = Reference(
sheet,
min_col=1,
min_row=2,
max_row=sheet.max_row
)
e.add_data(f, titles_from_data=True)
e.set_categories(g)
e.title = "Sales Chart"
sheet.add_chart(e, "Q2")
wb1.save("sales_updated.xlsx")


Activities
Google Chrome
Sep 7 3:25 PM
Home
Home
Untitled
answers.pdf
VIT Vellore - VTOP
localhost:8888/notebooks/Untitled.ipynb
jupyter Untitled Last Checkpoint: 59 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
6. Using Pandas: a. Load the Excel sheet to a data frame
[63]:
import pandas as pd
a = pd.read_excel("stu.xlsx")
b. Inspect the data with df.head(), df.info(), df.describe()
[62]:
print(a.head())
a.info()
print(a.describe())
name age marks grade
0 vickey 12 98 A
1 narmatha 12 97 A
2 ajay 23 96 A
3 priya 23 95 A
4 sonu 33 97 B
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
Column Non-Null Count Dtype
0 name 10 non-null str
1 age 10 non-null int64
2 marks 10 non-null int64
3 grade 10 non-null str
dtypes: int64(2), str(2)
memory usage: 514.0 bytes
age marks
count 10.000000 10.000000
mean 22.800000 92.500000
std 11.516172 8.154753
min 12.000000 72.000000
25% 12.000000 91.750000
50% 23.000000 95.500000
75% 29.750000 97.000000
max 46.000000 99.000000
c. Print the data by a specific Column name
[65]:
print(a["age"])
0 12
1 12



c. Print the data by a specific Column name

```
[65]: print(a["age"])
```

```
0    12
1    12
2    23
3    23
4    33
5    32
6    12
7    12
8    23
9    46
Name: age, dtype: int64
```

d. Print the data from the first five rows

```
[66]: print(a.head(5))
```

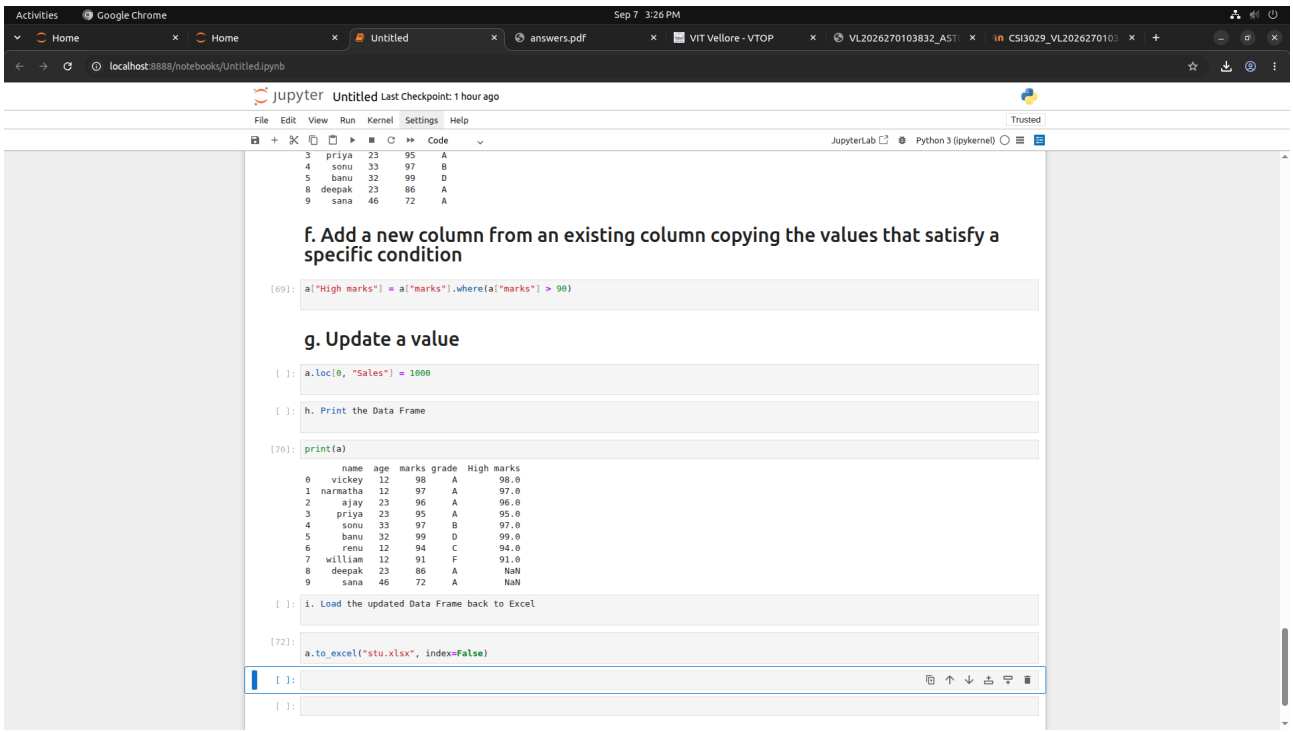
```
      name  age  marks grade
0   vickey   12     98     A
1  narmatha  12     97     A
2     ajay   23     96     A
3    priya   23     95     A
4     sonu   33     97     B
```

e. Filter the data in a specific column with a specific condition

```
[67]: b = a[a["age"] > 20]
print(b)
```

```
     name  age  marks grade
2    ajay   23     96     A
3   priya   23     95     A
4    sonu   33     97     B
5    banu   32     99     D
8  deepak   23     86     A
9    sana   46     72     A
```

f. Add a new column from an existing column copying the values that satisfy a



3 priya 23 95 A
4 sonu 33 97 B
5 banu 32 99 D
8 deepak 23 86 A
9 sana 46 72 A

f. Add a new column from an existing column copying the values that satisfy a specific condition

```
[69]: a["High marks"] = a["marks"].where(a["marks"] > 90)
```

g. Update a value

```
[ ]: a.loc[0, "Sales"] = 1000
```

```
[ ]: h. Print the Data Frame
```

```
[70]: print(a)
```

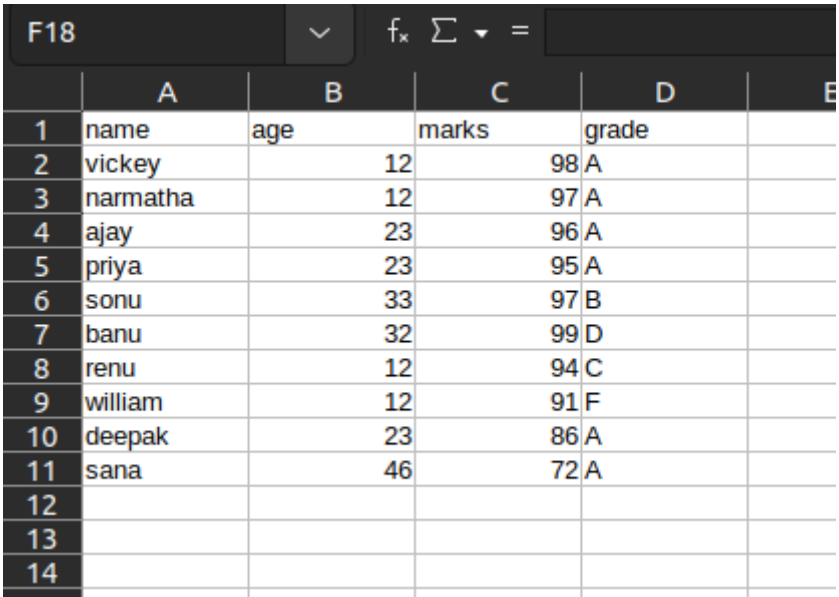
```
      name  age  marks grade  High marks
0   vickey   12     98     A        98.0
1  narmatha  12     97     A        97.0
2     ajay   23     96     A        96.0
3    priya   23     95     A        95.0
4     sonu   33     97     B        97.0
5     banu   32     99     D        99.0
6     renu   12     94     C        94.0
7  william   12     91     F        91.0
8   deepak   23     86     A         NaN
9     sana   46     72     A         NaN
```

```
[ ]: i. Load the updated Data Frame back to Excel
```

```
[72]: a.to_excel("stu.xlsx", index=False)
```

```
[ ]:
```

```
[ ]:
```



F18

	A	B	C	D
1	name	age	marks	grade
2	vickey	12	98	A
3	narmatha	12	97	A
4	ajay	23	96	A
5	priya	23	95	A
6	sonu	33	97	B
7	banu	32	99	D
8	renu	12	94	C
9	william	12	91	F
10	deepak	23	86	A
11	sana	46	72	A
12				
13				
14				

10. Series Practice - Part 1

```
import pandas as pd
import numpy as np
```

a. Create a simple series of Student_Names and Programmes they belong with

```
Student_Names = ['Arun', 'Bala', 'Charu', 'Divya', 'Eshwar']
Programmes = ['B.Sc CS', 'B.Com', 'BBA', 'B.Sc Maths', 'BCA']

student_series = pd.Series(Programmes, index=Student_Names)
print(student_series)
```

```
Arun          B.Sc CS
Bala            B.Com
Charu             BBA
Divya     B.Sc Maths
Eshwar            BCA
dtype: object
```

```
student_series = pd.Series(data=Programmes, index=Student_Names,
name='Programme', dtype=object)
student_series.index.name = 'Student_Name'
print(student_series)
```

```
Student_Name
Arun          B.Sc CS
Bala            B.Com
Charu             BBA
Divya     B.Sc Maths
Eshwar            BCA
Name: Programme, dtype: object
```

b. Create a series from a list, tuple, dictionary and a custom index

```
marks_list = [78, 85, 90, 66, 72]
series_from_list = pd.Series(marks_list)
print(series_from_list)
```

```
0    78
1    85
2    90
3    66
4    72
dtype: int64
```

```
marks_tuple = (78, 85, 90, 66, 72)
series_from_tuple = pd.Series(marks_tuple)
print(series_from_tuple)
```

```
0    78
1    85
2    90
3    66
4    72
dtype: int64
```

```
marks_dict = {'Arun': 78, 'Bala': 85, 'Charu': 90, 'Divya': 66,
'Eshwar': 72}
series_from_dict = pd.Series(marks_dict)
print(series_from_dict)
```

```
Arun      78
Bala      85
Charu     90
Divya     66
Eshwar    72
dtype: int64
```

```
series_custom_index = pd.Series(marks_list, index=['S1', 'S2', 'S3',
'S4', 'S5'])
print(series_custom_index)
```

```
S1    78
S2    85
S3    90
S4    66
S5    72
dtype: int64
```

c. Create two series with CAT I and CAT II marks of students and explore element-wise operations

```
students = ['Arun', 'Bala', 'Charu', 'Divya', 'Eshwar']

cat1 = pd.Series([45, 38, 50, 29, 41], index=students, name='CAT_I')
cat2 = pd.Series([48, 42, 46, 35, 39], index=students, name='CAT_II')

print(cat1)
print()
print(cat2)
```

```
Arun      45
Bala      38
Charu     50
Divya     29
```

```
Eshwar    41
Name: CAT_I, dtype: int64

Arun      48
Bala      42
Charu     46
Divya     35
Eshwar    39
Name: CAT_II, dtype: int64
```

```
total = cat1 + cat2
difference = cat2 - cat1
average = (cat1 + cat2) / 2
product = cat1 * 2
percentage = (total / 100) * 100

print(total)
print()
print(difference)
print()
print(average)
print()
print(product)
print()
print(percentage)
print()
print(cat2 > cat1)
```

```
Arun      93
Bala      80
Charu     96
Divya     64
Eshwar    80
dtype: int64

Arun      3
Bala      4
Charu    -4
Divya     6
Eshwar   -2
dtype: int64

Arun      46.5
Bala      40.0
Charu     48.0
Divya     32.0
Eshwar    40.0
dtype: float64

Arun       90
```

```
Bala       76
Charu     100
Divya      58
Eshwar     82
Name: CAT_I, dtype: int64

Arun      93.0
Bala      80.0
Charu     96.0
Divya     64.0
Eshwar    80.0
dtype: float64

Arun       True
Bala       True
Charu     False
Divya      True
Eshwar    False
dtype: bool
```

d. Write conditions to check if data in a series are integers/chars/Not a Number

```
mixed = pd.Series([10, 'A', 25.5, np.nan, 'Bala', 7, None, '99'])
print(mixed)

0      10
1       A
2    25.5
3     NaN
4    Bala
5       7
6    None
7      99
dtype: object
```

Error cell

```
print(mixed.isnumeric())

---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/var/folders/l1/rlv1r3x571n8gf_bgmt84mmr0000gn/T/ipykernel_45464/615965805.py in ?()
----> 1 print(mixed.isnumeric())

/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
```

```
site-packages/pandas/core/generic.py in ?(self, name)
   6314             and name not in self._accessors
   6315             and
self._info_axis._can_hold_identifiers_and_holds_name(name)
   6316         ):
   6317             return self[name]
-> 6318         return object.__getattribute__(self, name)

AttributeError: 'Series' object has no attribute 'isnumeric'
```

Corrected cell

```
is_integer = mixed.apply(lambda x: isinstance(x, (int, np.integer)))
is_char = mixed.apply(lambda x: isinstance(x, str))
is_nan = mixed.isna()

result = pd.DataFrame({
    'Value': mixed,
    'Is_Integer': is_integer,
    'Is_Char': is_char,
    'Is_NaN': is_nan
})
print(result)
```

	Value	Is_Integer	Is_Char	Is_NaN
0	10	True	False	False
1	A	False	True	False
2	25.5	False	False	False
3	NaN	False	False	True
4	Bala	False	True	False
5	7	True	False	False
6	None	False	False	True
7	99	False	True	False

```
for i in mixed.index:
    value = mixed[i]
    if pd.isna(value):
        print(i, ':', value, '--> Not a Number')
    elif isinstance(value, (int, np.integer)):
        print(i, ':', value, '--> Integer')
    elif isinstance(value, str):
        print(i, ':', value, '--> Character/String')
    else:
        print(i, ':', value, '--> Float/Other')
```

```
0 : 10 --> Integer
1 : A --> Character/String
2 : 25.5 --> Float/Other
3 : nan --> Not a Number
4 : Bala --> Character/String
```

```
5 : 7 --> Integer
6 : None --> Not a Number
7 : 99 --> Character/String
```

e. Create a series for Product Names, Prices, Quantities in a retail store

```
product_names = ['Soap', 'Shampoo', 'Toothpaste', 'Rice 5kg', 'Sugar
1kg']

prices = pd.Series([35.50, 180.00, 95.75, 420.00, 48.25],
index=product_names, name='Price')
quantities = pd.Series([120, 45, 80, 30, 150], index=product_names,
name='Quantity')

print(prices)
print()
print(quantities)
```

```
Soap           35.50
Shampoo       180.00
Toothpaste     95.75
Rice 5kg      420.00
Sugar 1kg      48.25
Name: Price, dtype: float64

Soap          120
Shampoo        45
Toothpaste     80
Rice 5kg       30
Sugar 1kg     150
Name: Quantity, dtype: int64
```

```
stock_value = prices * quantities
stock_value.name = 'Stock_Value'

print(stock_value)
print()
print('Total stock value :', stock_value.sum())
print('Costliest product :', prices.idxmax())
print('Highest quantity  :', quantities.idxmax())
```

```
Soap           4260.0
Shampoo        8100.0
Toothpaste     7660.0
Rice 5kg      12600.0
Sugar 1kg      7237.5
Name: Stock_Value, dtype: float64

Total stock value : 39857.5
```

```
Costliest product : Rice 5kg
Highest quantity  : Sugar 1kg
```

f. Create a Multilevel series with arrays, tuple, product and dataframe

```
sem = ['Sem1', 'Sem1', 'Sem1', 'Sem2', 'Sem2', 'Sem2']
subject = ['Maths', 'Physics', 'Chemistry', 'Maths', 'Physics',
'Chemistry']

idx_arrays = pd.MultiIndex.from_arrays([sem, subject],
names=['Semester', 'Subject'])
ml_arrays = pd.Series([88, 76, 91, 82, 79, 85], index=idx_arrays)
print(ml_arrays)
```

```
Semester  Subject
Sem1      Maths        88
          Physics      76
          Chemistry    91
Sem2      Maths        82
          Physics      79
          Chemistry    85
dtype: int64
```

Using tuples

```
tuples = [('Sem1', 'Maths'), ('Sem1', 'Physics'), ('Sem1',
'Chemistry'),
          ('Sem2', 'Maths'), ('Sem2', 'Physics'), ('Sem2',
'Chemistry')]

idx_tuples = pd.MultiIndex.from_tuples(tuples, names=['Semester',
'Subject'])
ml_tuples = pd.Series([88, 76, 91, 82, 79, 85], index=idx_tuples)
print(ml_tuples)
```

```
Semester  Subject
Sem1      Maths        88
          Physics      76
          Chemistry    91
Sem2      Maths        82
          Physics      79
          Chemistry    85
dtype: int64
```

```
idx_product = pd.MultiIndex.from_product([['Sem1', 'Sem2'], ['Maths',
'Physics', 'Chemistry']],
                                         names=['Semester', 'Subject'])
ml_product = pd.Series([88, 76, 91, 82, 79, 85], index=idx_product)
print(ml_product)
```

```
Semester  Subject
Sem1      Maths        88
          Physics      76
          Chemistry    91
Sem2      Maths        82
          Physics      79
          Chemistry    85
dtype: int64
```

Using dataframe

```
df = pd.DataFrame({
    'Semester': ['Sem1', 'Sem1', 'Sem1', 'Sem2', 'Sem2', 'Sem2'],
    'Subject': ['Maths', 'Physics', 'Chemistry', 'Maths', 'Physics',
'Chemistry']
})

idx_frame = pd.MultiIndex.from_frame(df)
ml_frame = pd.Series([88, 76, 91, 82, 79, 85], index=idx_frame)
print(ml_frame)
```

```
Semester  Subject
Sem1      Maths        88
          Physics      76
          Chemistry    91
Sem2      Maths        82
          Physics      79
          Chemistry    85
dtype: int64
```

g. Access the data elements using First Level Index and Second Level Index

```
print(ml_product['Sem1'])
print()
print(ml_product.loc['Sem2'])
print()
print(ml_product.loc[:, 'Physics'])
print()
print(ml_product.xs('Chemistry', level='Subject'))
print()
print(ml_product.xs('Sem1', level='Semester'))
```

```
Subject
Maths        88
Physics      76
Chemistry    91
dtype: int64
```

```
Subject
Maths        82
Physics      79
Chemistry    85
dtype: int64

Semester
Sem1    76
Sem2    79
dtype: int64

Semester
Sem1    91
Sem2    85
dtype: int64

Subject
Maths        88
Physics      76
Chemistry    91
dtype: int64
```

h. Access a specific data item using .loc

Error cell

```
print(ml_product.loc['Sem1', 'Physics', 'Maths'])

---------------------------------------------------------------------------
IndexingError                             Traceback (most recent call last)
Cell In[22], line 1
----> 1 print(ml_product.loc['Sem1', 'Physics', 'Maths'])

File /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas/core/indexing.py:1184, in _LocationIndexer.__getitem__(self, key)
   1182     if self._is_scalar_access(key):
   1183         return self.obj._get_value(*key, takeable=self._takeable)
-> 1184     return self._getitem_tuple(key)
   1185 else:
   1186     # we by definition only have the 0th axis
   1187     axis = self.axis or 0

File /Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packages/pandas/core/indexing.py:1371, in
```

```
_LocIndexer._getitem_tuple(self, tup)
   1368     return self._getitem_lowerdim(tup)
   1370 # no multi-index, so validate all of the indexers
-> 1371 tup = self._validate_tuple_indexer(tup)
   1373 # ugly hack for GH #836
   1374 if self._multi_take_opportunity(tup):

File
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
site-packages/pandas/core/indexing.py:962, in
_LocationIndexer._validate_tuple_indexer(self, key)
    957 @final
    958 def _validate_tuple_indexer(self, key: tuple) -> tuple:
    959     """
    960     Check the key for valid keys across my indexer.
    961     """
--> 962     key = self._validate_key_length(key)
    963     key = self._expand_ellipsis(key)
    964     for i, k in enumerate(key):

File
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/
site-packages/pandas/core/indexing.py:1001, in
_LocationIndexer._validate_key_length(self, key)
    999             raise IndexingError(_one_ellipsis_message)
   1000         return self._validate_key_length(key)
-> 1001     raise IndexingError("Too many indexers")
   1002 return key

IndexingError: Too many indexers
```

Corrected cell

```
print(ml_product.loc['Sem1', 'Physics'])
print()
print(ml_product.loc[('Sem2', 'Chemistry')])
print()
print(ml_product.loc[[('Sem1', 'Maths'), ('Sem2', 'Physics')]])
print()
print(prices.loc['Shampoo'])
print()
print(cat1.loc['Charu'])
```

```
76

85

Semester  Subject
Sem1      Maths      88
Sem2      Physics    79
```

```
dtype: int64

180.0

50
```